

Отчёт по задаче VRP

Ломаченко Олег, ФПМИ, 3 курс

1 Постановка задачи

Задача VRP формулируется следующим образом. Имеется один склад (точка 0), $N-1$ клиентов, V машин и вместимость каждой машины c . Каждая машина стартует со склада, посещает группу клиентов и возвращается обратно. Требуется:

- каждый клиент посещается ровно один раз;
- суммарный спрос клиентов на маршруте одной машины не превышает c ;
- минимизировать суммарную длину всех маршрутов.

2 Как все начиналось

К сожалению, я пропустил семинар по VRP, поэтому пришлось самостоятельно изучать классические подходы.

2.1 Старты

Savings-эвристика Clarke–Wright [1] : каждый клиент сначала в отдельном маршруте $0 \rightarrow i \rightarrow 0$, потом маршруты объединяются по величине экономии $d(0, i) + d(0, j) - d(i, j)$ с проверкой ограничения c . Эксперименты показали, что этот старт особенно полезен на тесте `vrp_101_10_1`, где sweep давал плохое значение.

Sweep-эвристика Gillett–Miller [3]: клиенты сортируются по полярному углу относительно склада, формируются маршруты, проверяется вместимость. Sweep хорошо работает для секторной геометрии, особенно на тестах `vrp_26_8_1` и `vrp_51_5_1`.

Тут надо понимать что задача трудная даже на уровне поиска стартового допустимого решения. В моем представлении стартовая эвристика должна давать допустимое. Сначала я реализовал Savings, потому что они попроще и это приятнее чем геометрия. Но к сожалению на `vrp_200_16_1` старт был не допустим. Я закодил старт через Sweep но там было тоже два теста которые не допустимые, но среди них не было `vrp_200_16_1`. По сему я решил что раз они дополняют друг друга, то сделать два этих старта.

Каждый старт на тех тестах где был допустимых проходил слабый порог :)

2.2 Локальный поиск

Для улучшения качества маршрутов я реализовал локальные операции:

- **2-opt** [2]: разворот сегмента маршрута внутри одной машины, что устраняет плохие локальные перемены.
- **Relocate** и **Swap**: перемещение и обмен клиентов между маршрутами при сохранении ограничений c .

Это дало сильные пороги кроме `vrp_16_3_1`, `vrp_26_8_1`, `vrp_421_41_1`. Я попробовал поиграться с параметрами но не помогло. Тогда я посмотрел в сторону LNS.

2.3 LNS

Это стандартный подход, но тут важно как идет восстановление. Часть клиентов удаляется, затем вставляется обратно через regret insertion, клиенты с высоким regret вставляются раньше, чтобы минимизировать ухудшение. Regret insertion — это эвристика вставки клиентов в маршруты, которая учитывает не только стоимость вставки в наилучшее место, но и разницу между лучшей и второй лучшей позицией вставки. Для каждого клиента i вычисляем ее и при построении маршрутов выбираем для вставки клиента с наибольшим regret, потому что если отложить его вставку, потом для него останется только плохое место, что сильно увеличит суммарную стоимость маршрутов.

LNS позволил существенно улучшить результаты на `vrp_16_3_1`, `vrp_26_8_1`, `vrp_421_41_1`.

3 Вывод

Старты очень имбовые эвристики, мне вообще казалось там ничего не построится потому что эта задача какой то очень тяжелый вариант SubSetSum подобных задач, но видимо сочетание того что точки на плоскости и что все же задача не искусственно выкрученная спасают ситуацию. Отдельно кстати хорошо себя показал LNS, именно тут это работает хорошо.

Список литературы

- [1] G. Clarke and J. W. Wright. Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4):568–581, 1964.
- [2] G. A. Croes. A method for solving traveling-salesman problems. *Operations Research*, 6(6):791–812, 1958.
- [3] Billy E. Gillett and Leland R. Miller. A heuristic algorithm for the vehicle-dispatch problem. *Operations Research*, 22(2):340–349, 1974.